Assignment-II

Smira Jaitley(241020)

Part I: Short Answer Questions

Q1:

The count comes back as 0 because SQL treats NULL as an unknown value rather than as zero or an empty string. Comparing phone_number = NULL is really asking whether one unknown value matches another unknown value, and that comparison resolves to UNKNOWN rather than TRUE. Since WHERE only retains rows that evaluate to TRUE, every row gets dropped.

A corrected version of the query:

```
SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;
```

More broadly, aggregate functions such as SUM and AVG skip over NULLs, and any direct comparison involving NULL fails to return TRUE or FALSE — it returns UNKNOWN instead, since there's nothing concrete to evaluate.

Q2:

Yes, the DISTINCT keyword is unnecessary here. Grouping by department already guarantees one row per department, so there are no duplicate rows left for DISTINCT to remove — it's simply redundant.

Q3:

When a LEFT JOIN produces more rows than an INNER JOIN, the extra rows represent customers who have never placed an order.

- INNER JOIN keeps only customers who have at least one matching order record.
- LEFT JOIN keeps every customer regardless of order history, padding the order columns with NULL for those without any orders.

Q4:

Grouping by department collapses every employee in that department into a single output row, and AVG(salary) is well-defined because it reduces multiple salaries to one number. employee_name, however, is neither part of the GROUP BY list nor wrapped in an aggregate function — with many employees folded into one row, SQL has no consistent way to choose which single name to display, so it raises an error instead of guessing.

A corrected version:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

Q5:

This fails because WHERE operates on individual rows before any grouping has happened, and aggregate functions like AVG only make sense after rows have been grouped. To filter on an

aggregated value, HAVING must be used instead, since it applies its condition after the groups have already been formed.

A corrected version:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6:

- Non-correlated example: finding products priced above the table-wide average price. That average is one constant value computed once from the entire table, so the subquery executes a single time.
- Correlated example: finding employees who earn more than the average salary within their own department. Here the relevant "average" is different for each employee being checked, so the subquery has to be re-evaluated once per row — which can hurt performance on large tables.

Part II: Long Answer Question

Part A — Customers who have never placed an order

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN "Order" o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Explanation: An INNER JOIN alone would only surface customers who already have orders. Using a LEFT JOIN instead keeps every customer in the result set, and filtering on o.order_id IS NULL then isolates the customers whose order columns came back empty — meaning they've never ordered.

Part B — Average order value per restaurant

```
SELECT ot.restaurant_id, AVG(ot.order_value) AS avg_order_value
FROM (
  SELECT o.order_id, o.restaurant_id, SUM(mi.price * oi.quantity) AS
order_value
  FROM "Order" o
  JOIN OrderItem oi ON o.order_id = oi.order_id
  JOIN MenuItem mi ON oi.item_id = mi.item_id
  GROUP BY o.order_id, o.restaurant_id
) AS ot
GROUP BY ot.restaurant_id
HAVING COUNT(*) > 5;
```

Explanation: The inner query first computes the total value of each individual order. The outer query then averages those per-order totals by restaurant. Doing it in two steps like this stops orders containing many low-cost items from distorting the final average.

Part C — Restaurants outperforming the platform-wide average

```
SELECT b.restaurant_id, b.avg_order_value
FROM (
  SELECT ot.restaurant_id, AVG(ot.order_value) AS avg_order_value
  FROM (
    SELECT o.order_id, o.restaurant_id, SUM(mi.price * oi.quantity) AS
order_value
    FROM "Order" o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.order_id, o.restaurant_id
  ) AS ot
  GROUP BY ot.restaurant_id
  HAVING COUNT(*) > 5
) AS b
WHERE b.avg_order_value > (
  SELECT AVG(all_orders.order_value)
  FROM (
    SELECT SUM(mi.price * oi.quantity) AS order_value
    FROM "Order" o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.order_id
  ) AS all_orders
);
```

Explanation: The platform-wide average is a single fixed value that's the same no matter which restaurant is being checked, so this subquery is non-correlated and only has to be evaluated once.

Part D — Best-selling menu item at each restaurant

```
SELECT restaurant_id, item_id, item_name, total_qty
FROM (
  SELECT mi.restaurant_id, mi.item_id, mi.item_name, SUM(oi.quantity) AS
total_qty,
         RANK() OVER (PARTITION BY mi.restaurant_id
                      ORDER BY SUM(oi.quantity) DESC) AS rnk
  FROM OrderItem oi
  JOIN MenuItem mi ON oi.item_id = mi.item_id
  GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
) AS ranked
WHERE rnk = 1;
```

Explanation: The query first totals up how many units of each item were sold at each restaurant, then applies the RANK() window function to order items within each restaurant. This beats using MAX() because it naturally handles ties and still keeps the item name available in the result.

Part III: Normalization

Functional Dependencies:

- student_id → student_name
- course_id → course_name, instructor_id
- instructor_id → instructor_name, instructor_office
- (student_id, course_id) → grade, enrollment_date

Is the table in 1NF? Yes , every column stores a single atomic value per row, which is exactly what First Normal Form requires.

2NF violation:

The composite primary key is (student_id, course_id), but some attributes — such as student_name — depend on only student_id, not the full key. That's a partial dependency. As a practical consequence, deleting the row of the only enrolled student in a course would wipe out the course's information too.

3NF violation:

There's a transitive chain here: course_id determines instructor_id, which in turn determines instructor_name. So instructor_name depends on the primary key only indirectly, through instructor_id. This means changing an instructor's office would require updating every single enrollment row linked to that instructor, opening the door to update anomalies.

Decomposition into 3NF:

- Student(student_id, student_name) — separated out since student_id determines student_name.
- Course(course_id, course_name, instructor_id) — separated out since course_id determines course_name and instructor_id.
- Instructor(instructor_id, instructor_name, instructor_office) — separated out since instructor_id determines instructor_name and instructor_office, eliminating the transitive dependency.
- Enrollment(student_id, course_id, grade, enrollment_date) — retains only the attributes that depend on the full composite key.

Part IV: Design Justification

LEFT JOIN versus NOT EXISTS:

Both approaches can correctly identify customers who never placed an order, but LEFT JOIN was chosen for a couple of reasons:

- Checking IS NULL makes the "no matching order" condition easy to read at a glance.
- NOT IN was avoided because a single NULL value sitting in the order table can silently break the comparison and cause the query to return nothing at all.

Normalization trade-offs:

Normalizing the Enrollment table removes anomalies, but it comes at the cost of more complex queries.

- Pros: it prevents duplicate data and the update errors that come with it.
- Cons: viewing a complete record — say, a student's name, course, grade, and instructor's office — now requires joining four separate tables, adding load to the database.